

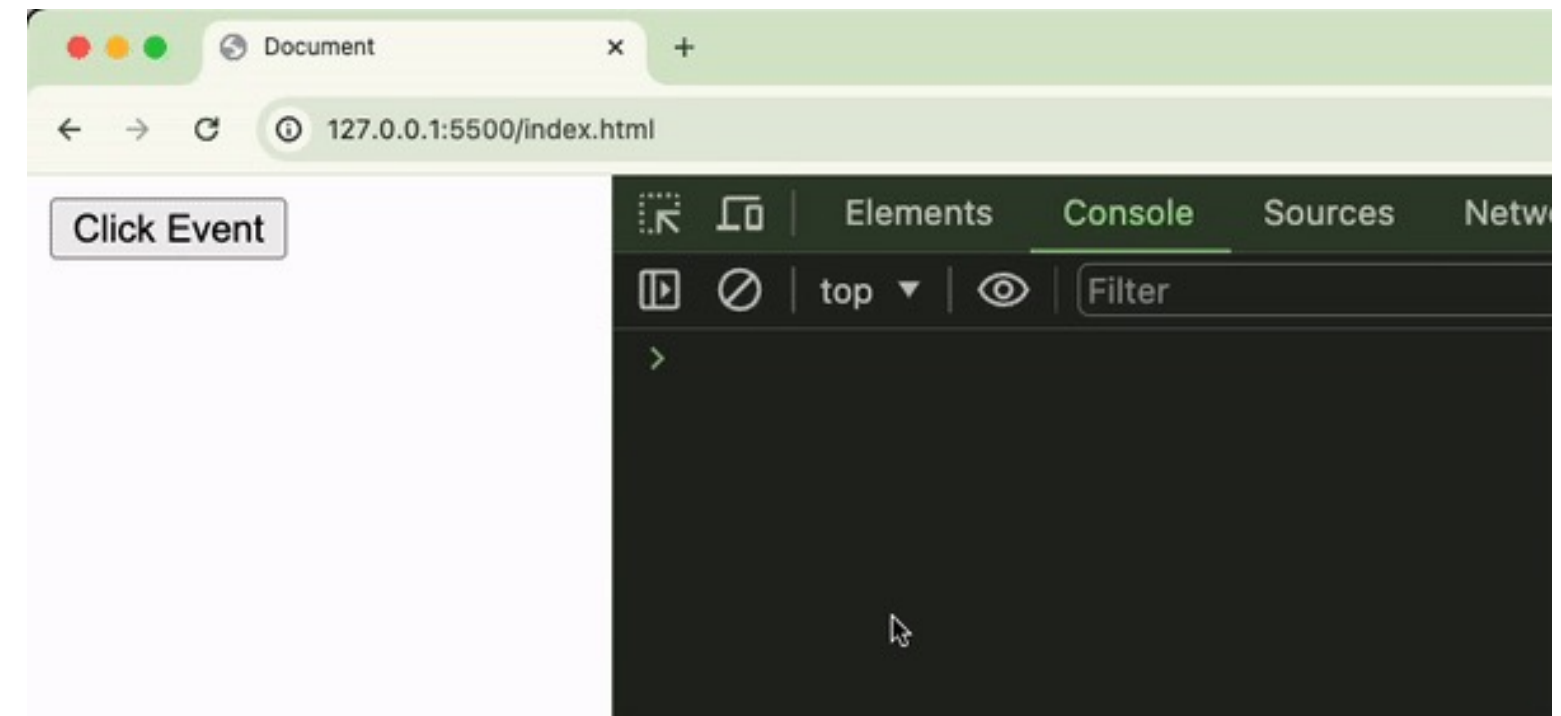
Web Basic

DOM과 이벤트

DOM과 이벤트 활용

이벤트 객체

```
var button = document.getElementById("btn");  
button.addEventListener("click", function(event) {  
    console.log(event);  
});
```



이벤트 핸들러 함수에 첫 번째 인수로 전달되는 객체
→ 해당 이벤트에 대한 정보와 기능이 담겨 있다.



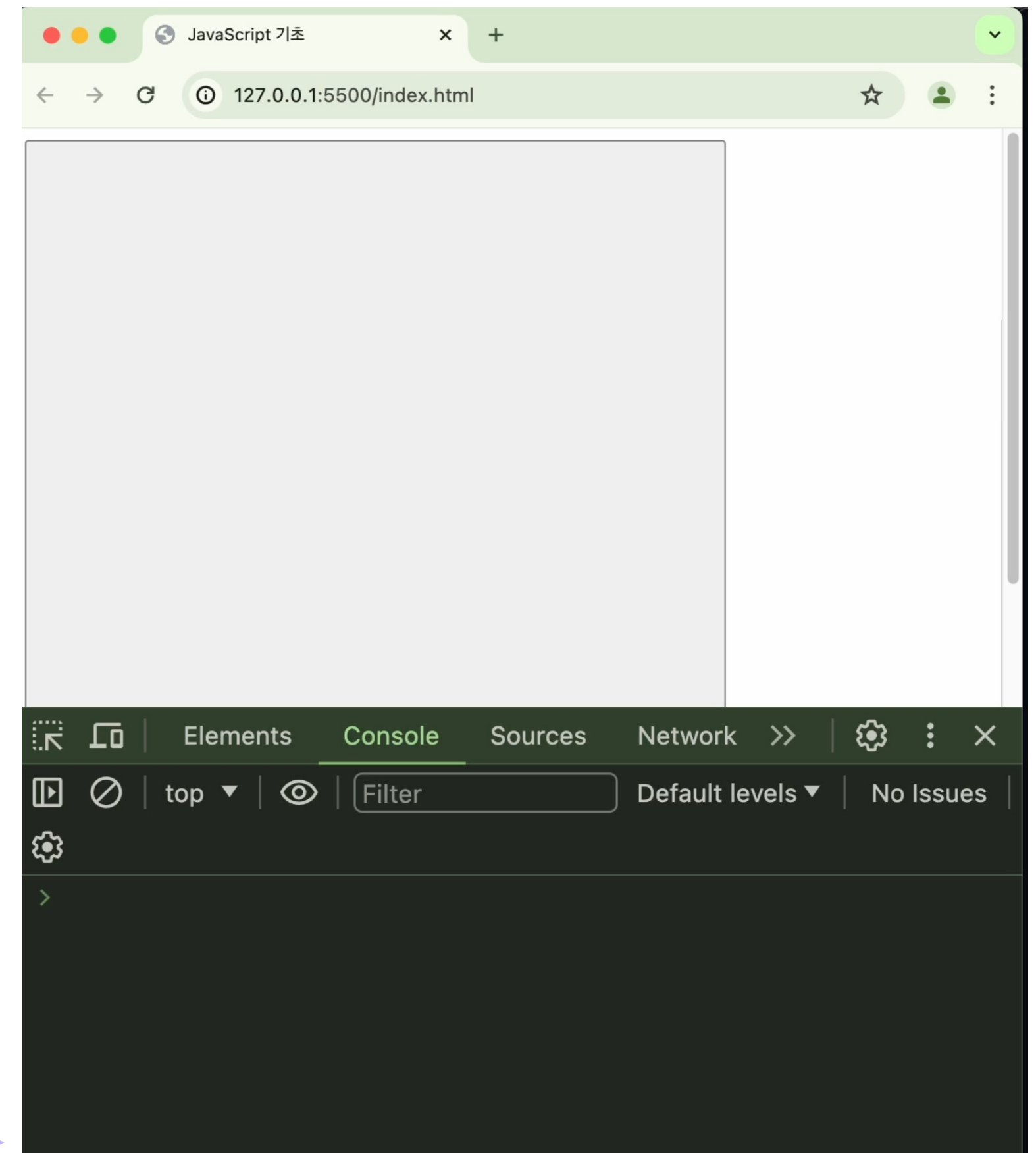
event 객체 - 마우스 정보

```
<button id="big-button" style="width: 500px; height: 500px"></button>

<script>
  var box = document.getElementById("big-button");

  box.addEventListener("click", function (event) {
    console.log(event.clientX, event.clientY);
  });
</script>
```

Play →





event 객체 - 키보드 정보

```
<h3>
  Pressed: "<span style="font-weight: 900; color: red" id="key"></span>"
</h3>

<script>
  window.addEventListener("keypress", function (event) {
    document.getElementById("key").innerText = event.key.toUpperCase();
  });
</script>
```



Pressed: ""

↑
Play

키보드 관련 이벤트의 경우 event 객체를 통해
키보드에 대한 정보를 얻을 수 있다. (누른 키 정보 등)

이처럼 **각 이벤트 발생에 대한 정보**를 event 객체를 통해 얻을 수 있다.



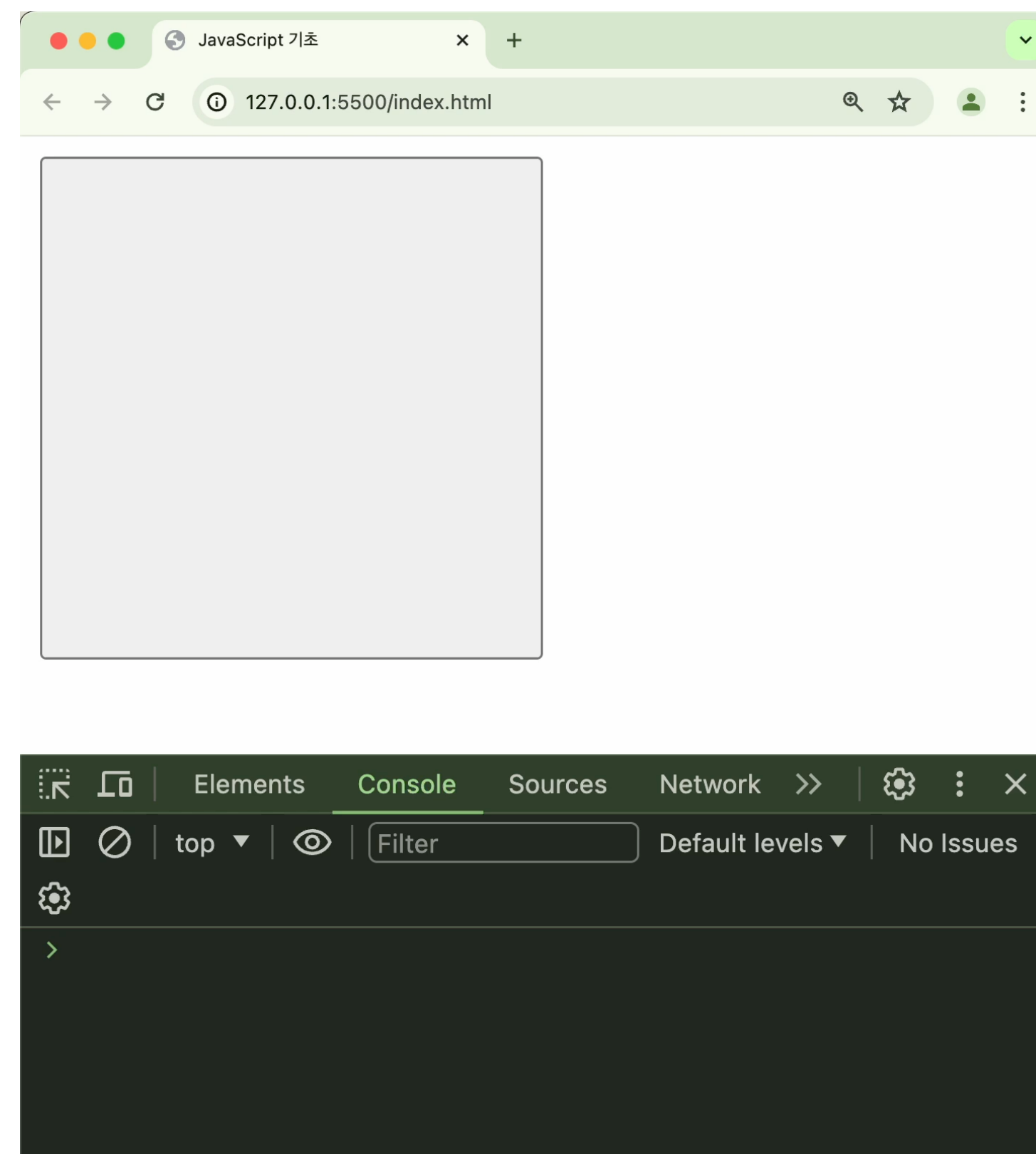
event 객체 – target

```
<button id="big-button" style="width: 200px; height: 200px"></button>

<script>
  var box = document.getElementById("big-button");

  box.addEventListener("click", function (event) {
    console.log(event.target);
  });
</script>
```

Play →



`event.target` 프로퍼티를 통해 이벤트가 발생한 타겟 요소의 node 객체에 접근할 수 있다.



event 객체 – preventDefault

```
<form id="login-form">
  <input type="text" placeholder="아이디" />
  <input type="password" placeholder="비밀번호" />
  <input type="submit" />
</form>
```

A screenshot of a web browser window titled "Document" with a single tab. The address bar shows the URL "127.0.0.1:5500/index.html". The page content displays a simple login form with two text input fields and a submit button. The first input field is labeled "아이디" (ID) and the second is labeled "비밀번호" (Password). The submit button is labeled "제출" (Submit).

form 태그에서 submit 버튼의 **기본 동작**은

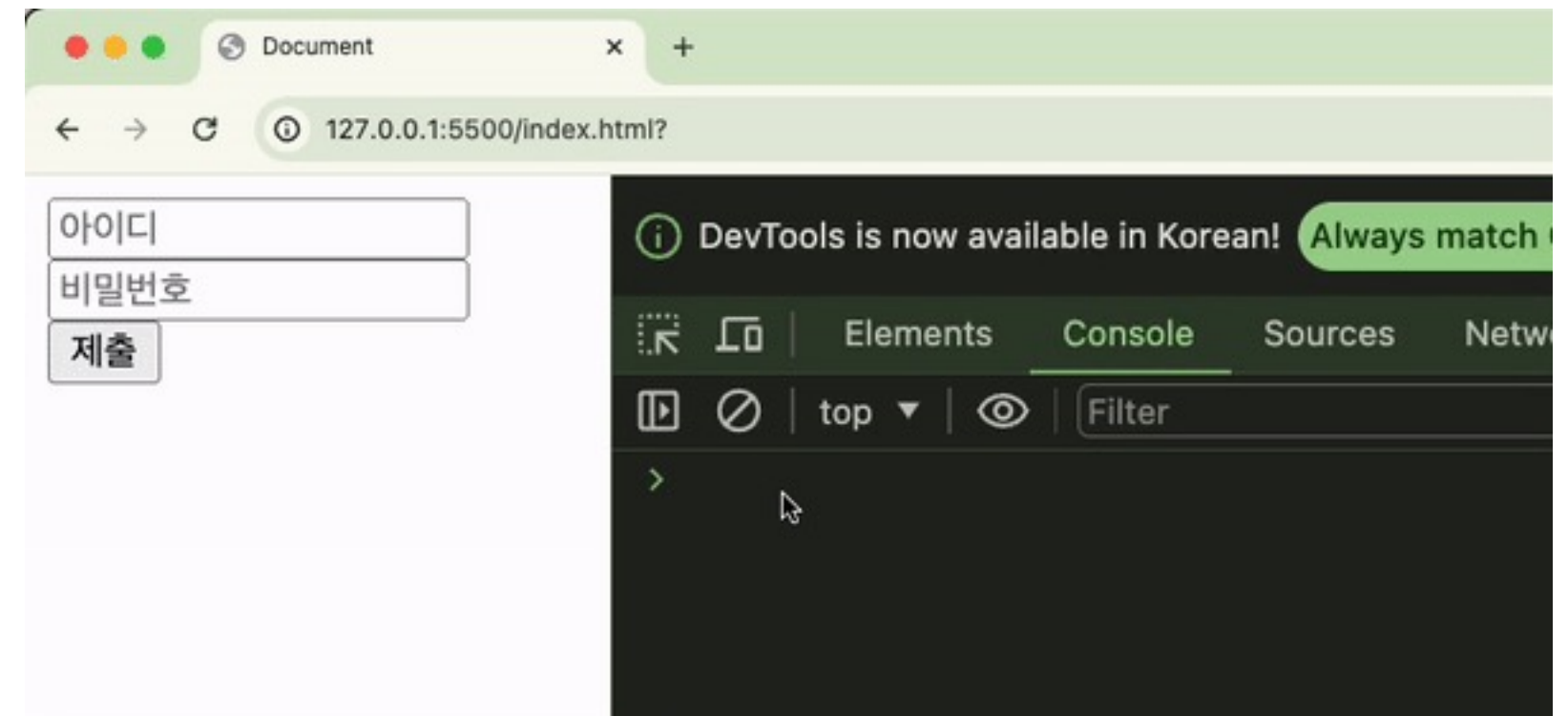
form 제출 → 새로고침



event 객체 – preventDefault

```
var loginForm = document.getElementById("login-form");
var idInput = document.getElementById("id");
var passwordInput = document.getElementById("password");

loginForm.addEventListener("submit", function (event) {
  event.preventDefault();
  console.log("아이디: ", idInput.value);
  console.log("비밀번호: ", passwordInput.value);
});
```



이 기본 동작을 막기 위해 event 객체의
event.preventDefault() 메서드를 사용한다.

이벤트 핸들러 제거 – removeEventListener

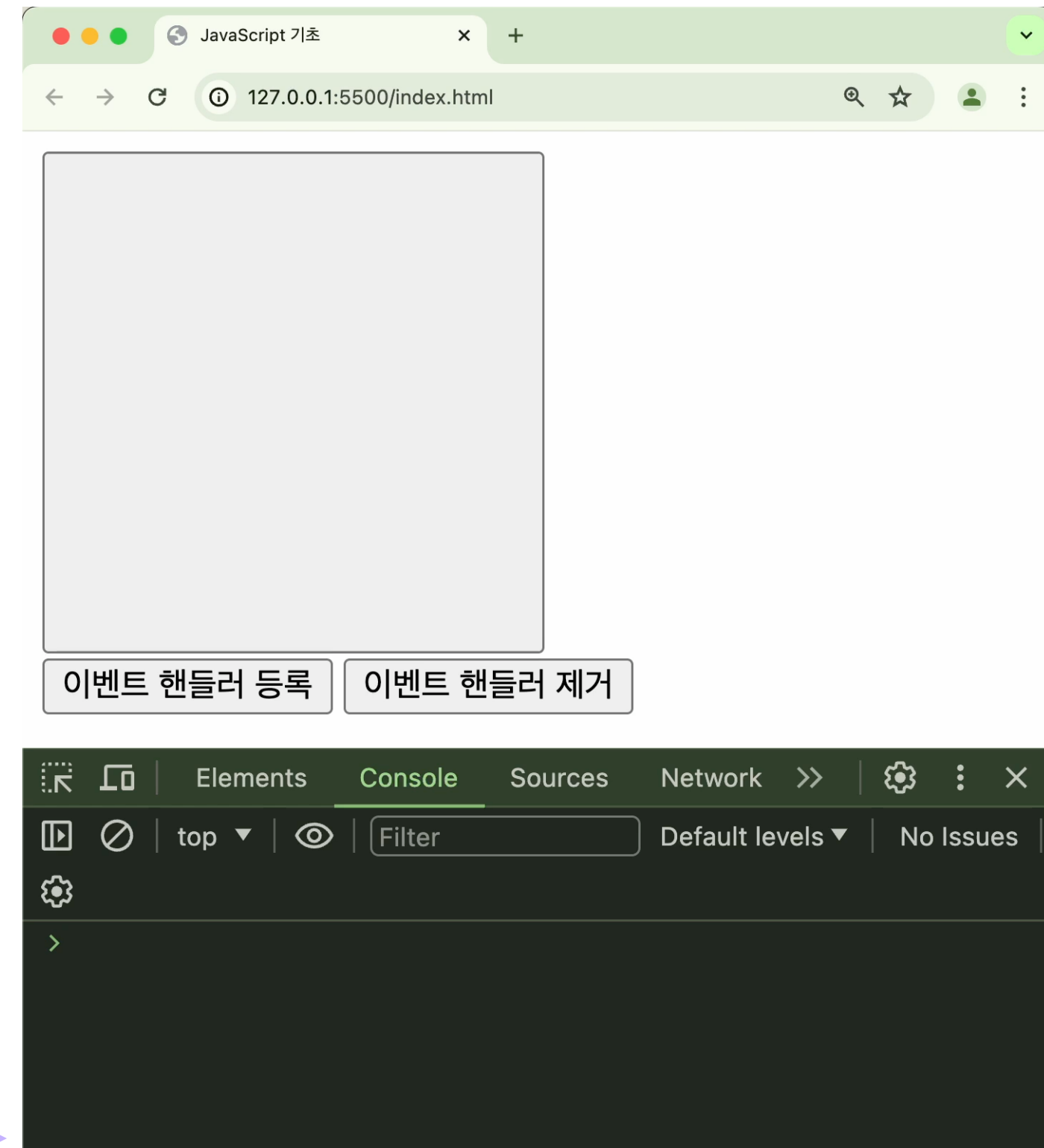
```
<button id="big-button" style="width: 200px; height: 200px"></button>
<br />
<button id="register">이벤트 핸들러 등록</button>
<button id="remove">이벤트 핸들러 제거</button>
<script>
  var box = document.getElementById("big-button");
  var registerBtn = document.getElementById("register");
  var removeBtn = document.getElementById("remove");

  function handleBtnClick(event) {
    console.log(event.clientX, event.clientY);
  }

  registerBtn.addEventListener("click", function () {
    box.addEventListener("click", handleBtnClick);
  });

  removeBtn.addEventListener("click", function () {
    box.removeEventListener("click", handleBtnClick);
  });
</script>
```

Play →

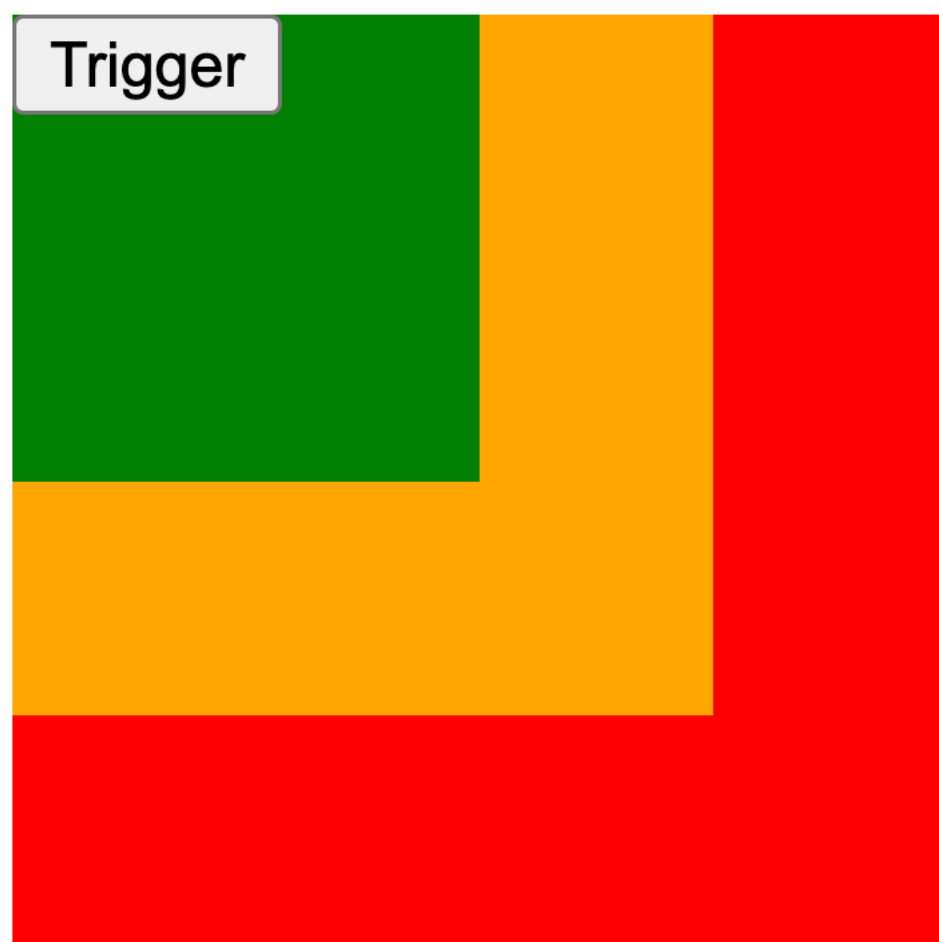
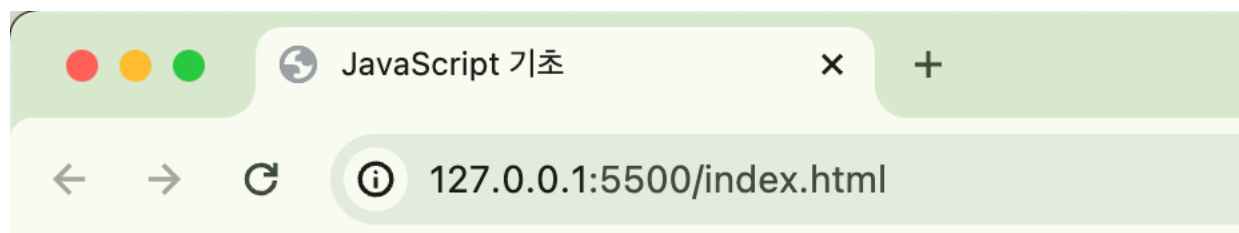


‘removeEventListener’ 메서드로 등록된 이벤트 핸들러를 제거할 수 있다.

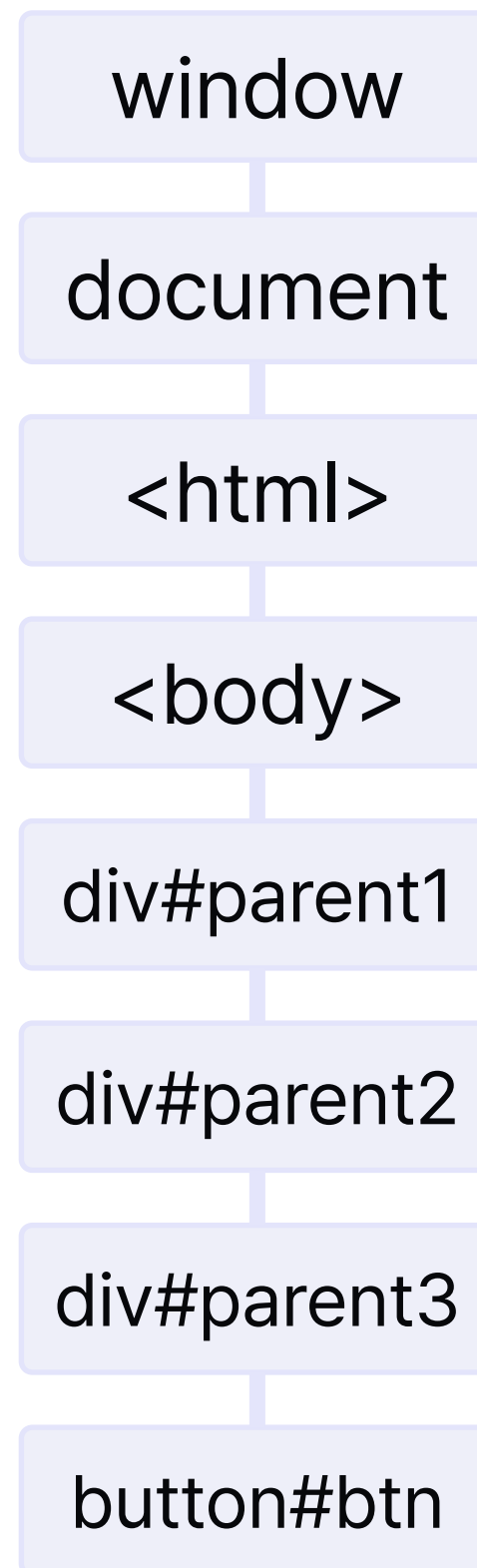
이벤트 propagation



이벤트 propagation



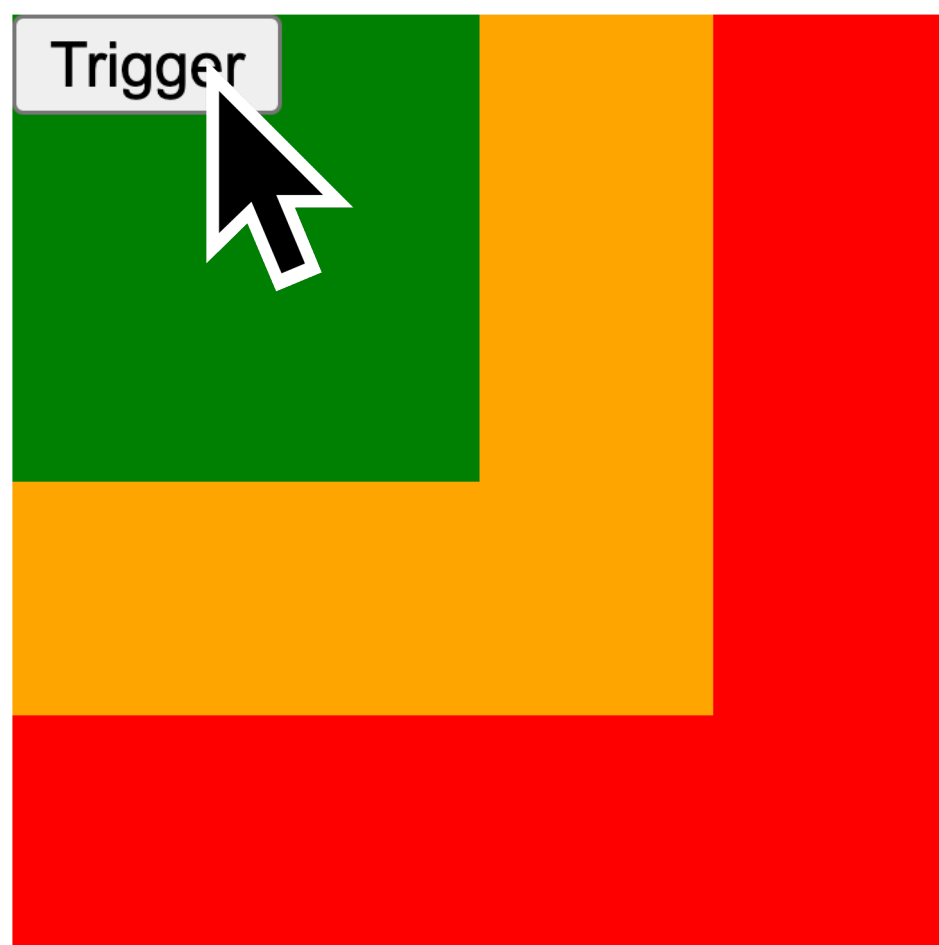
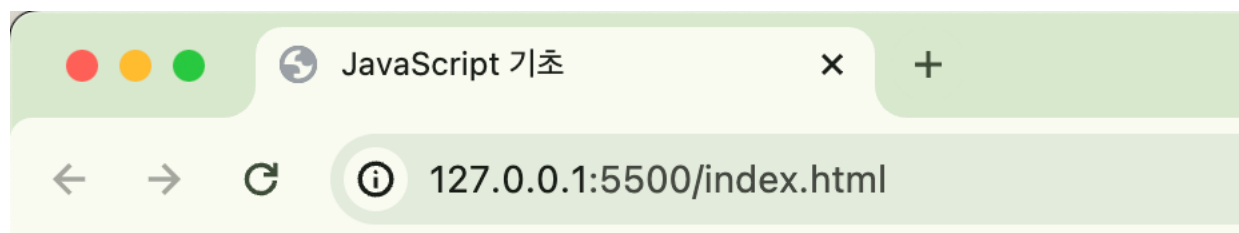
```
<div
  id="parent-1"
  style="width: 200px; height: 200px; background-color: red"
>
  <div
    id="parent-2"
    style="width: 150px; height: 150px; background-color: orange"
  >
    <div
      id="parent-3"
      style="width: 100px; height: 100px; background-color: green"
    >
      <button id="btn">Trigger</button>
    </div>
  </div>
</div>
```



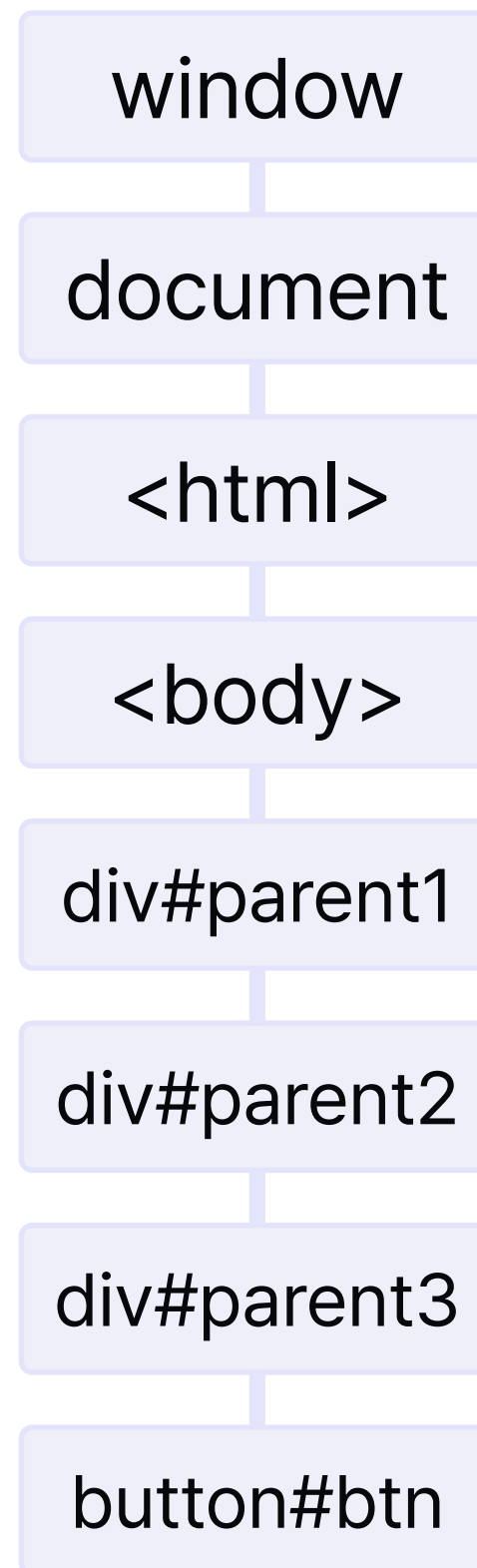
DOM에서 버튼 요소는 3개의 부모 요소와
그 상위 노드인 <body>, <html>, document, window 등이 있다.



이벤트 propagation



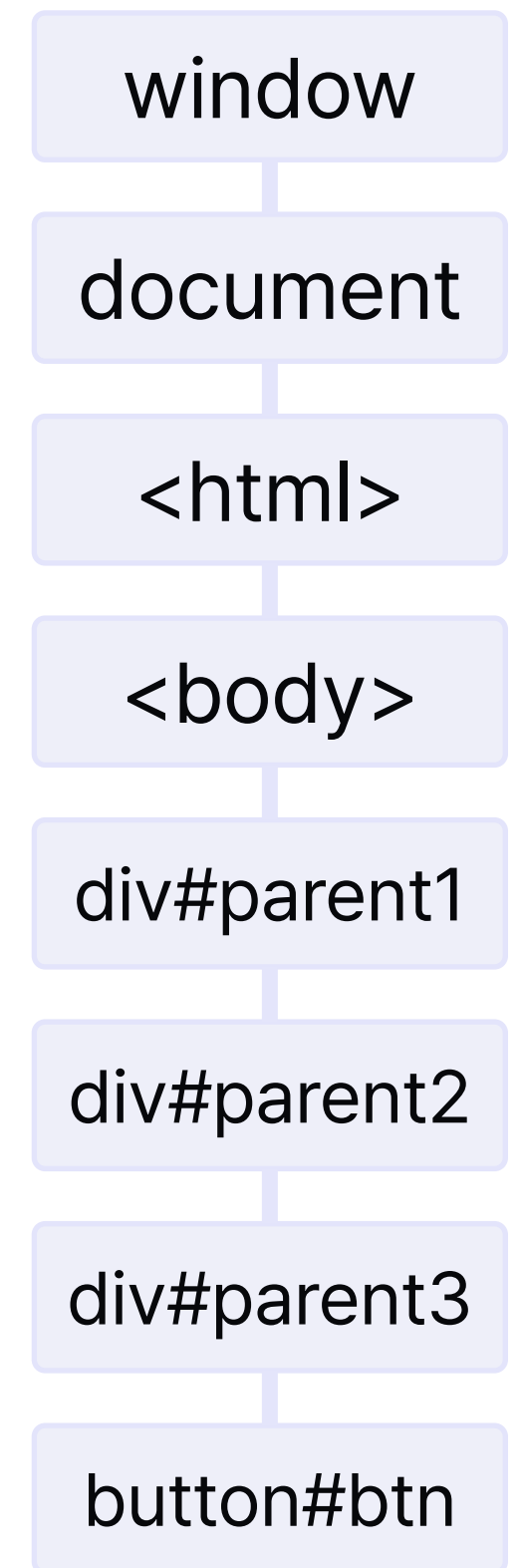
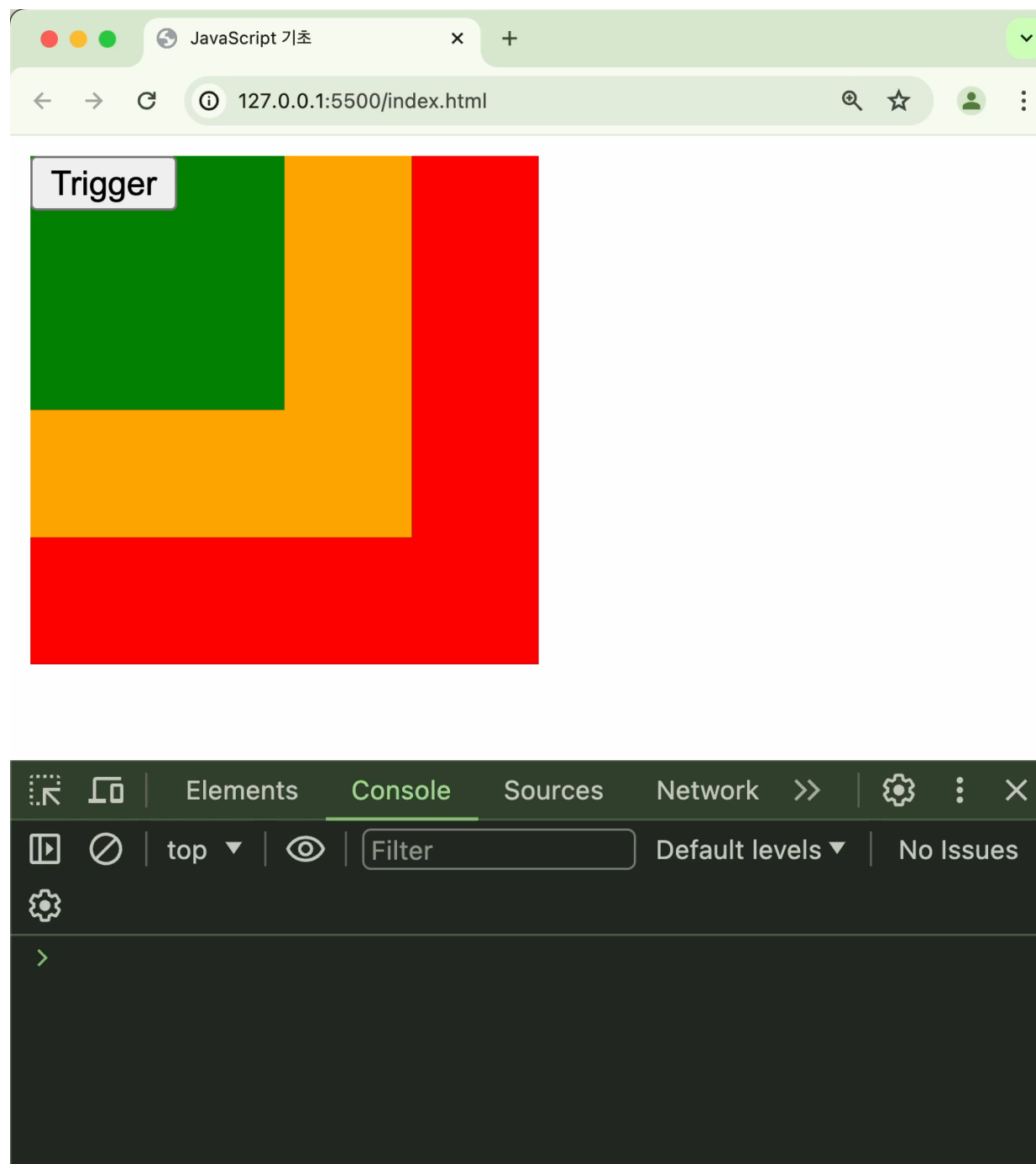
```
btn.addEventListener("click", function (event) { ... });  
parent3.addEventListener("click", function (event) { ... });  
parent2.addEventListener("click", function (event) { ... });  
parent1.addEventListener("click", function (event) { ... });  
...
```



가장 하위 요소인 버튼을 클릭하면, 이 이벤트는 어느 요소에서 발생하는 것일까?



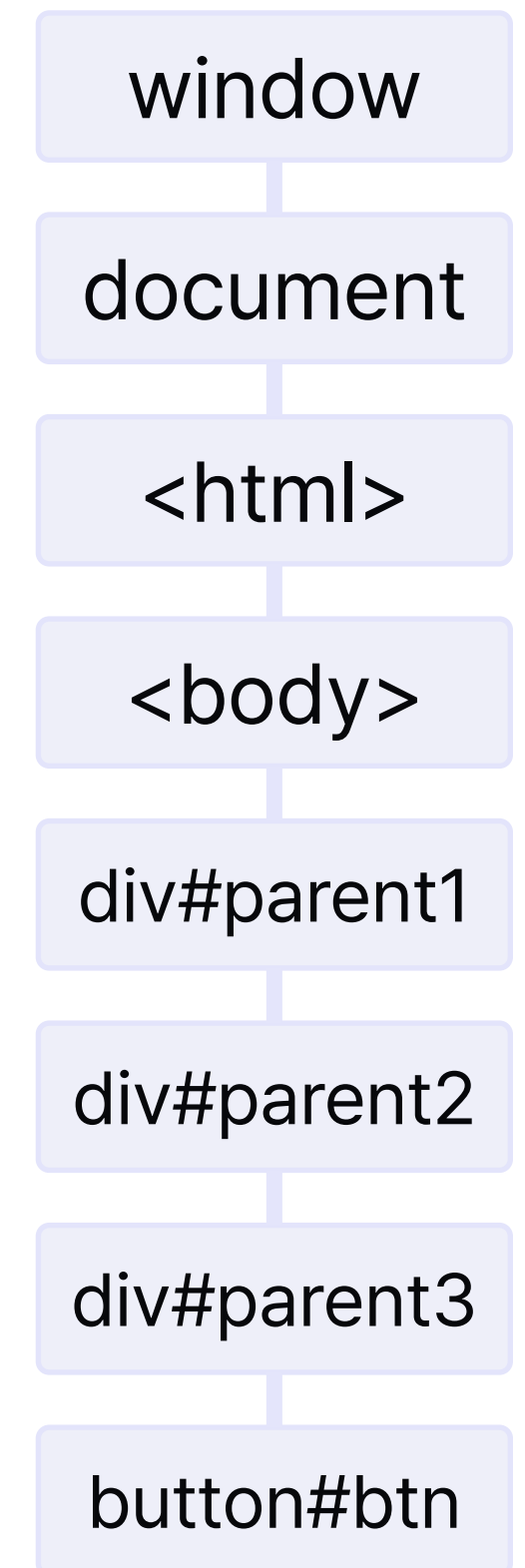
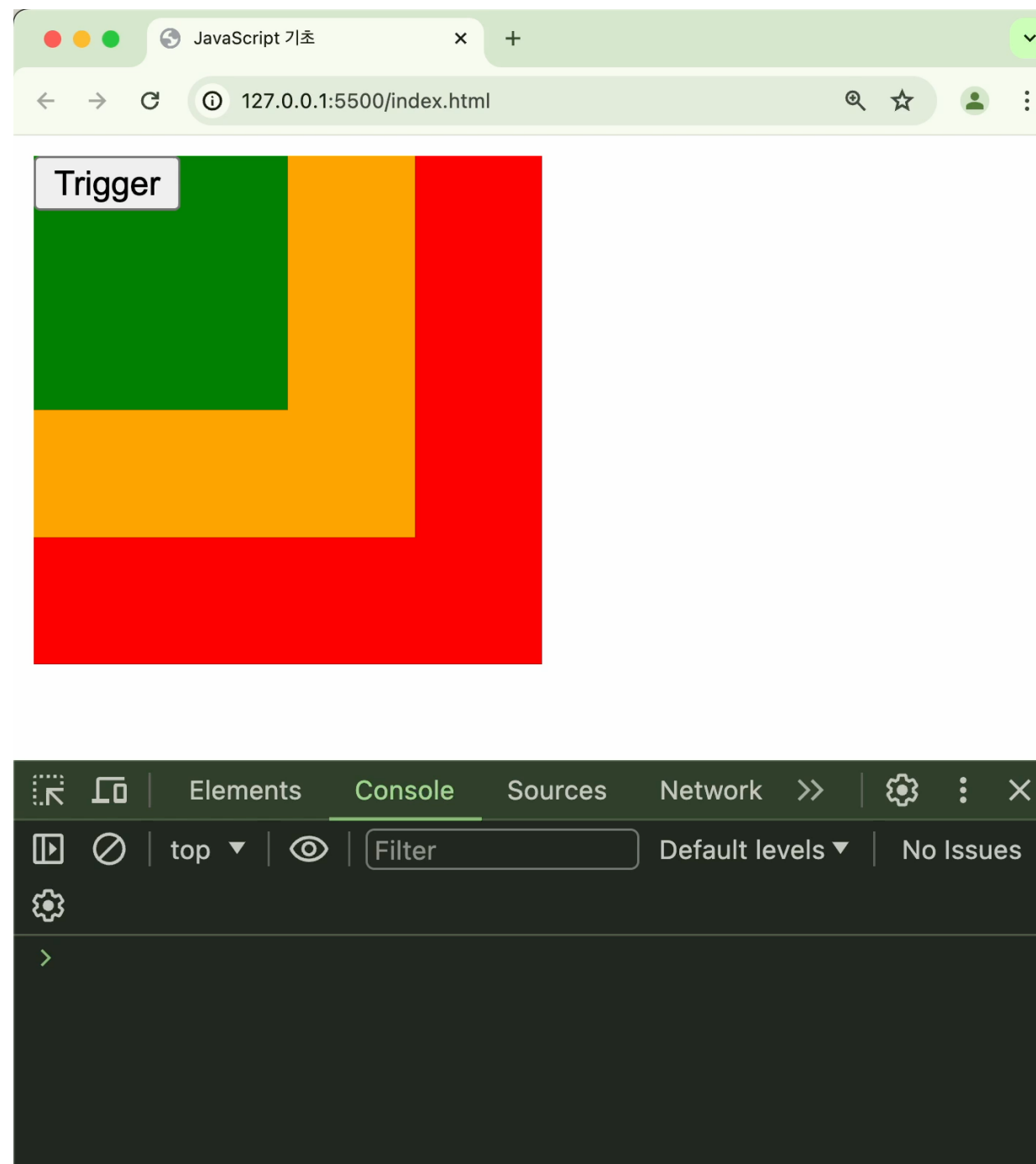
이벤트 propagation



정답은 모든 요소에서 이벤트가 발생한다!
= 모든 요소에서 이벤트를 캐치할 수 있다.



이벤트 propagation

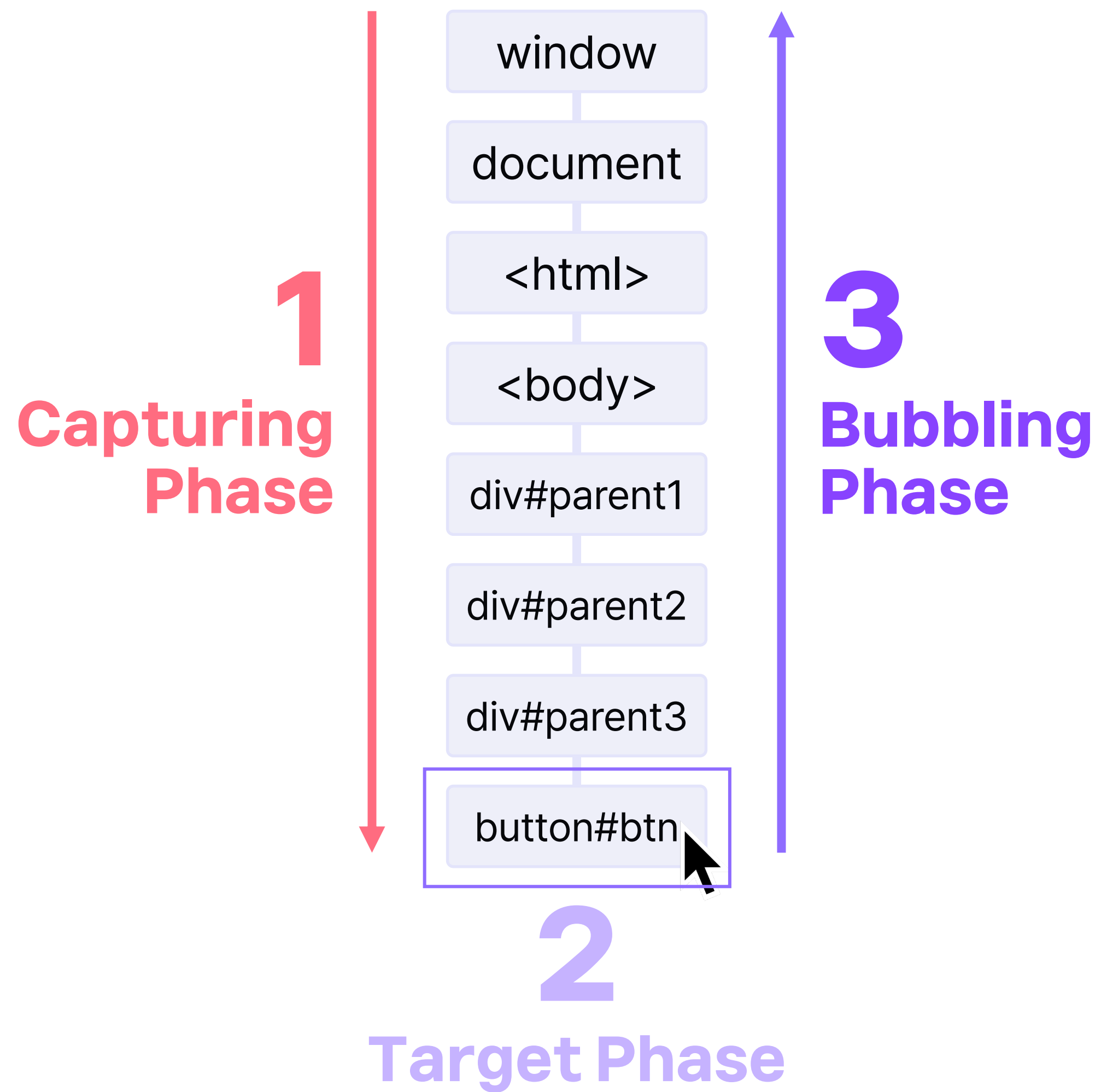


이벤트가 발생하면, 이 이벤트는 DOM 트리를 통해 전파된다.

이를 이벤트 propagation이라고 한다.

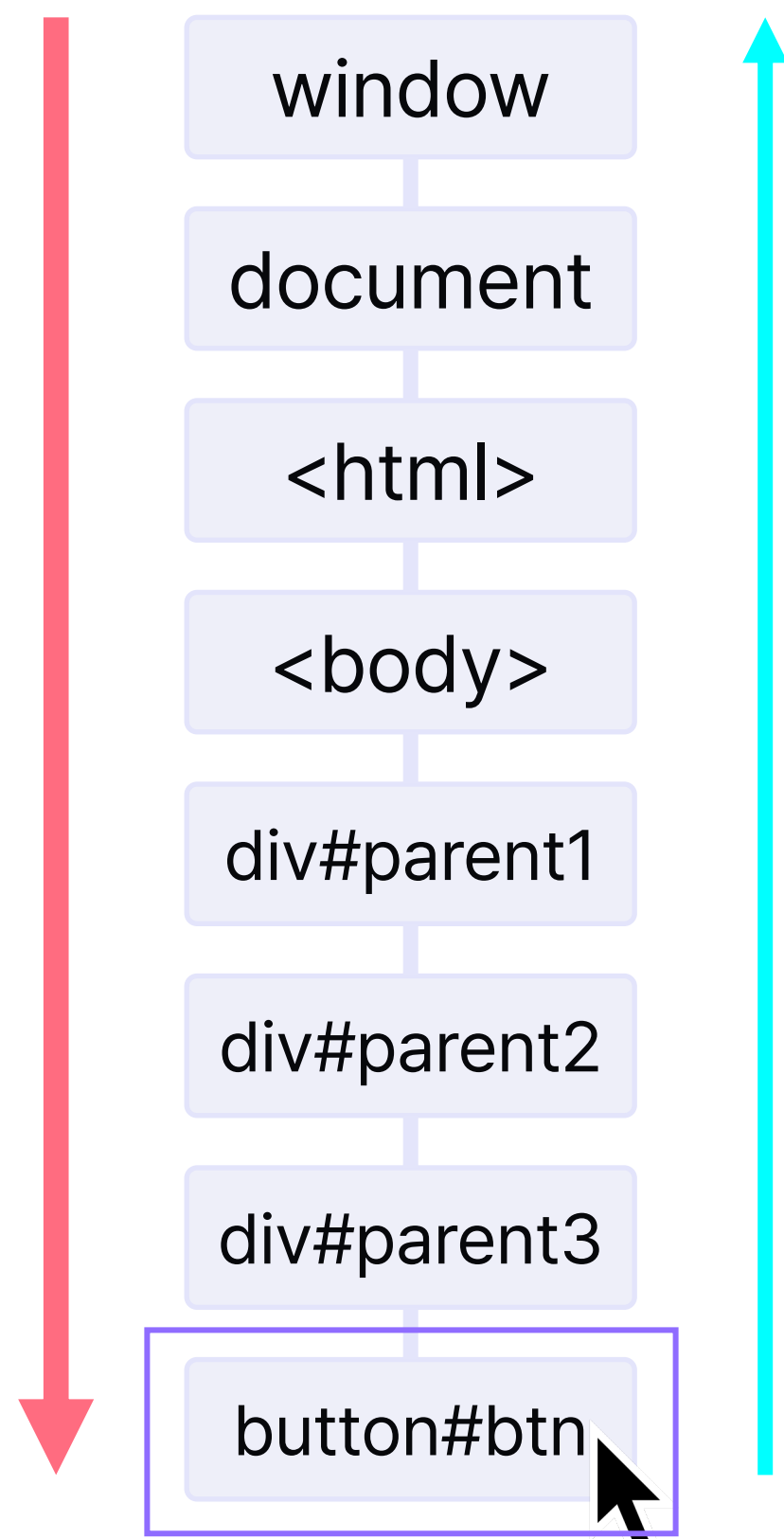


이벤트 propagation





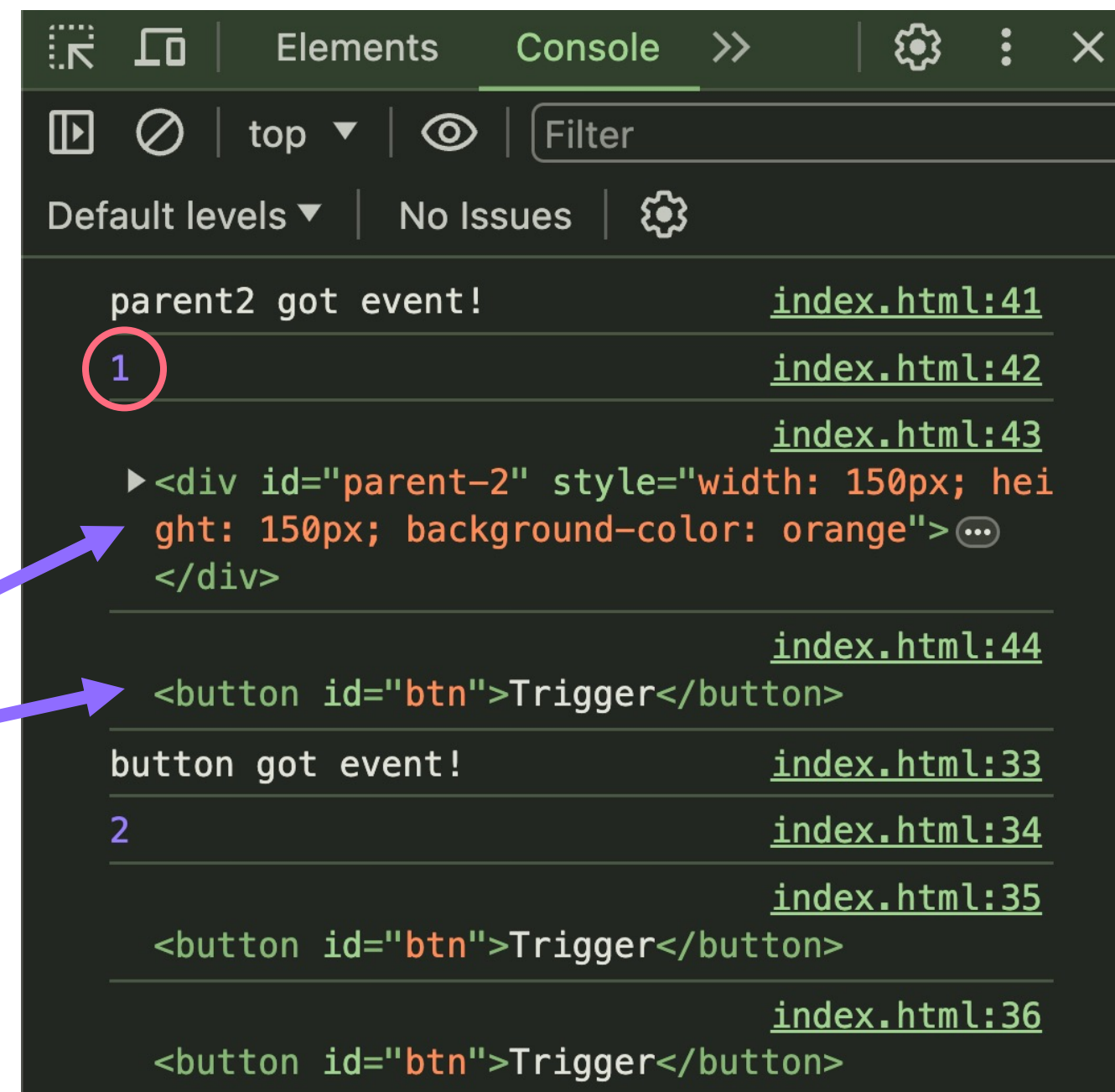
이벤트 propagation



1 Capturing Phase

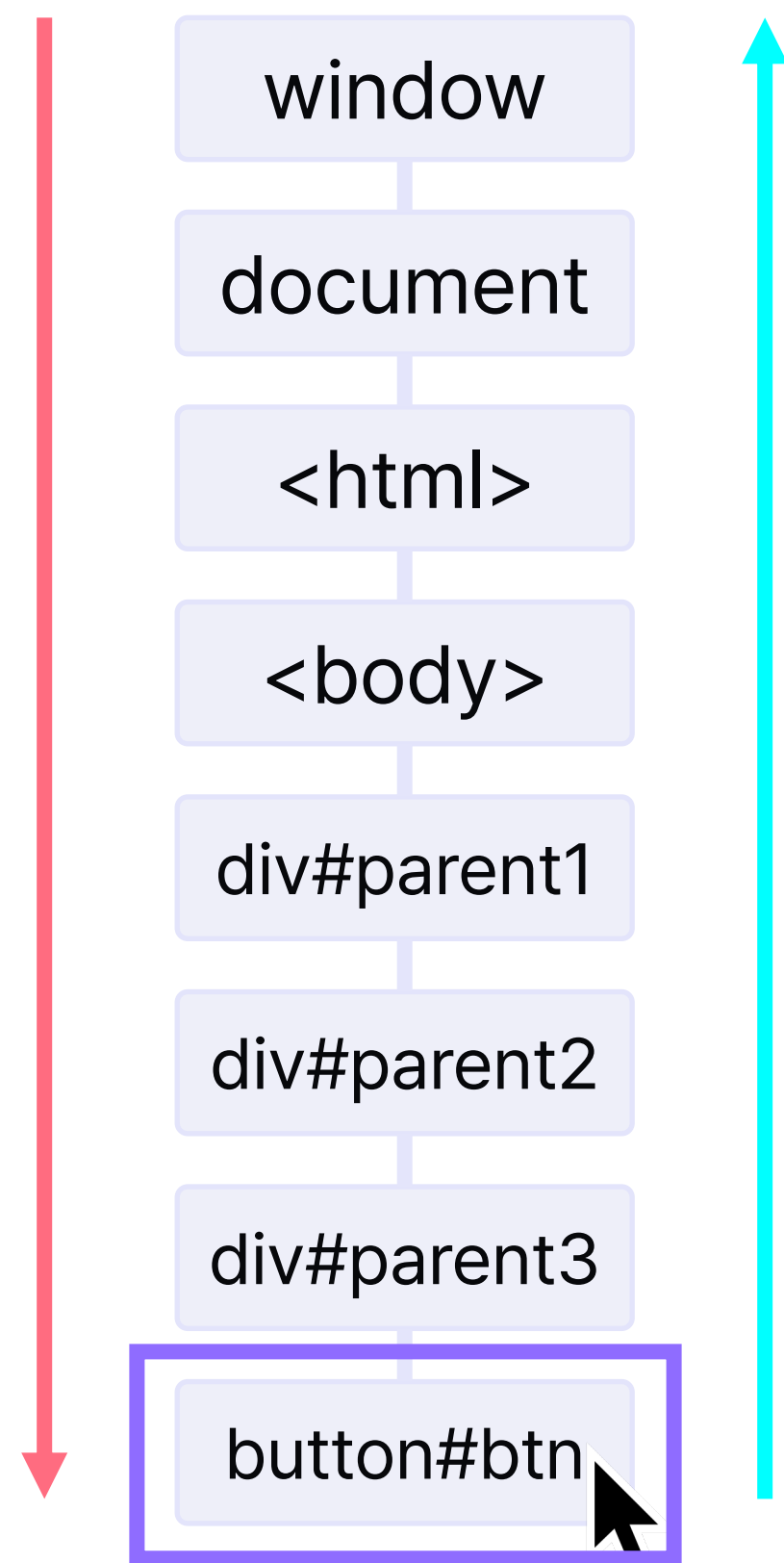
```
btn.addEventListener("click", function (event) {
  console.log("button got event!");
  console.log(event.eventPhase);
  console.log(event.currentTarget);
  console.log(event.target);
});
parent2.addEventListener(
  "click",
  function (event) {
    console.log("parent2 got event!");
    console.log(event.eventPhase);
    console.log(event.currentTarget);
    console.log(event.target);
  },
  true
);
```

Capturing phase에서 이벤트를 캐치하려면
addEventListener의 세 번째 인자로 `true` 전달 필요





이벤트 propagation



2 Target Phase

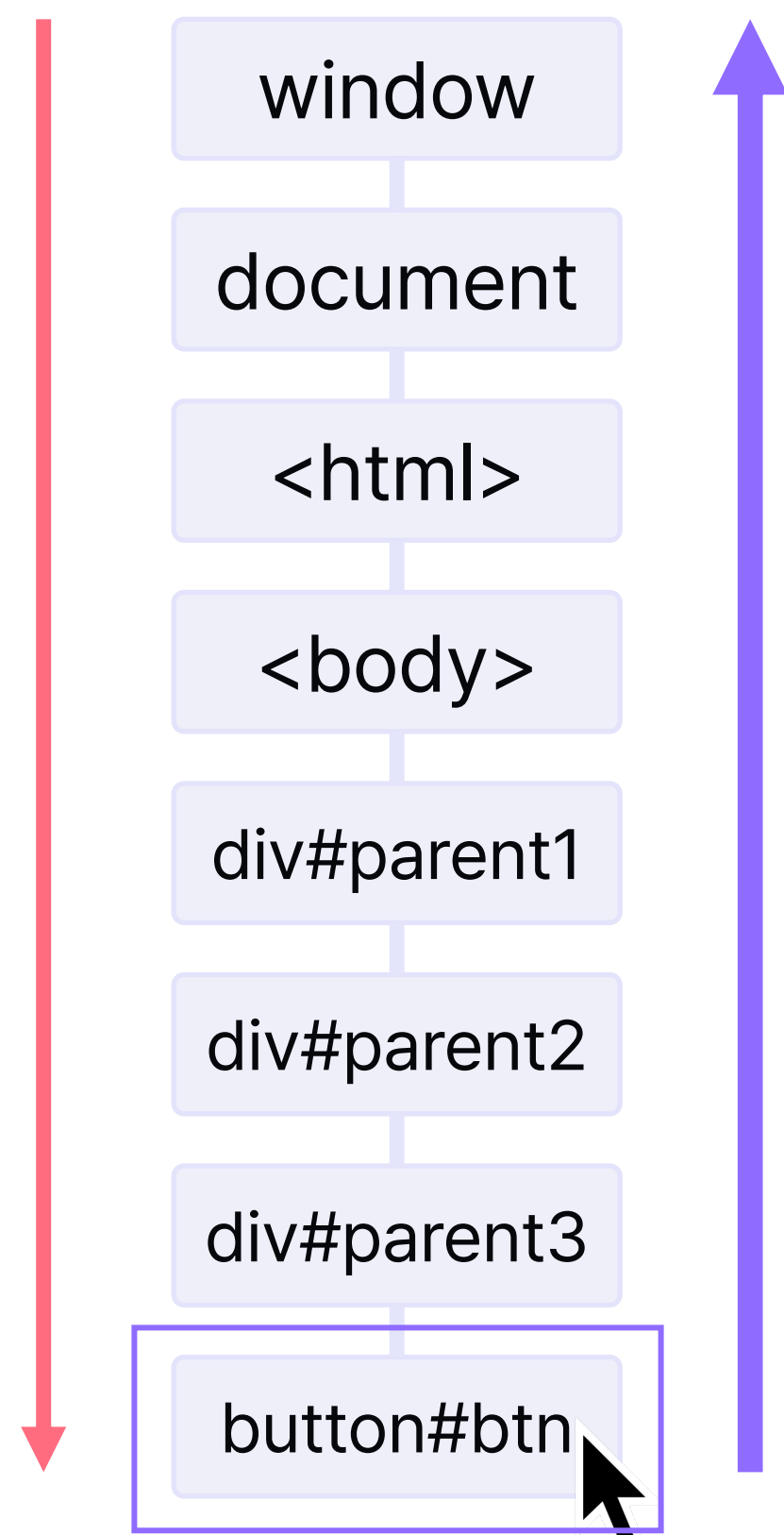
```
btn.addEventListener("click", function (event) {
  console.log("button got event!");
  console.log(event.eventPhase);
  console.log(event.currentTarget);
  console.log(event.target);
});
parent2.addEventListener(
  "click",
  function (event) {
    console.log("parent2 got event!");
    console.log(event.eventPhase);
    console.log(event.currentTarget);
    console.log(event.target);
  },
  true
);
```

A screenshot of a browser's developer console with the 'Console' tab selected. It shows a sequence of log messages during a click event on a button. The messages are: 'parent2 got event!' (index.html:41), '1' (index.html:42), a div element (index.html:43), '<button id="btn">Trigger</button>' (index.html:44), 'button got event!' (index.html:33), '2' (index.html:34), '<button id="btn">Trigger</button>' (index.html:35), and another '<button id="btn">Trigger</button>' (index.html:36). The number '2' is circled in red. Two purple arrows originate from the 'event.currentTarget' and 'event.target' logs in the code block above, pointing to the corresponding log entries in the console.



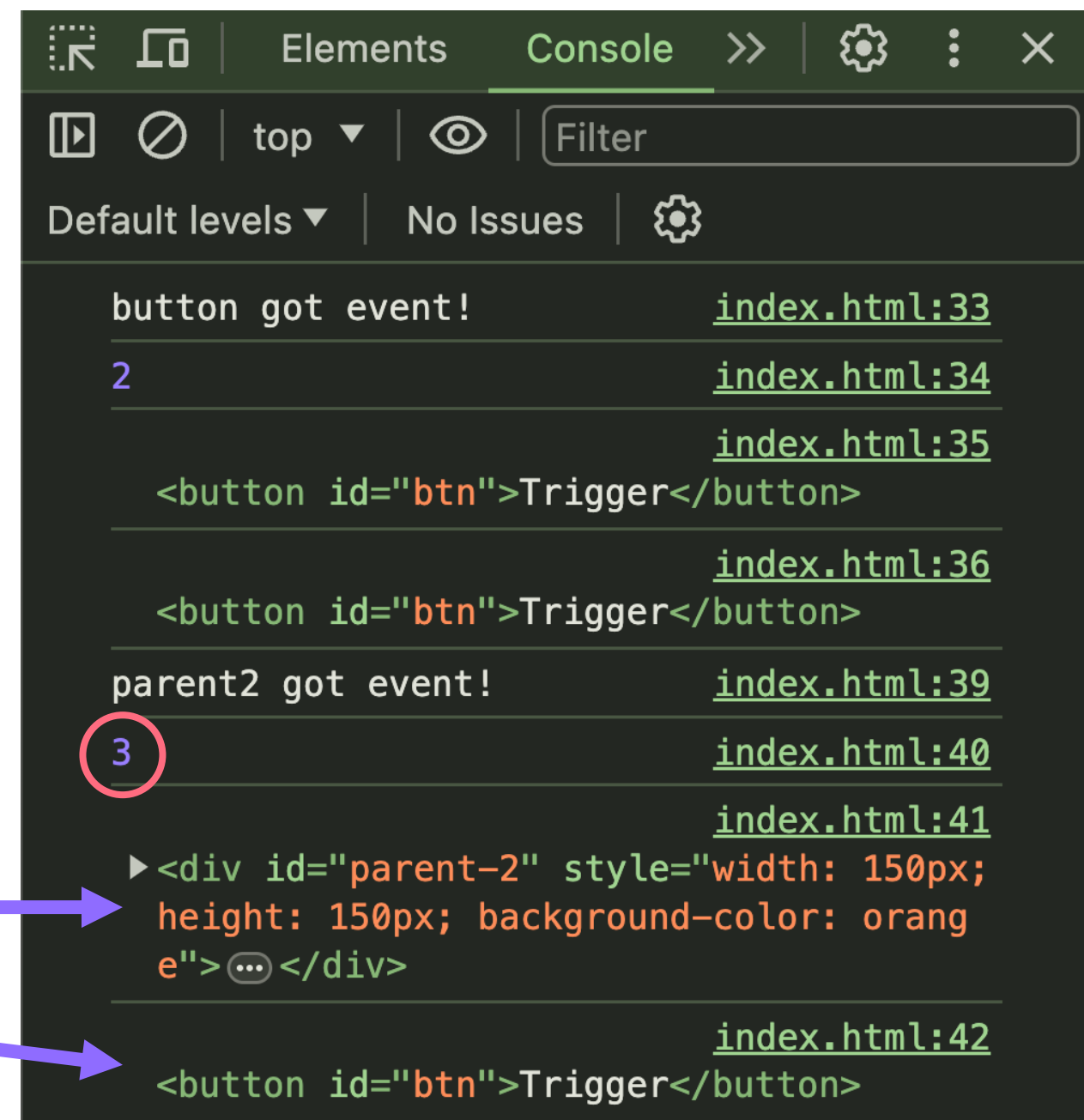
이벤트 propagation

3 Bubbling Phase



```
btn.addEventListener("click", function (event) {
  console.log("button got event!");
  console.log(event.eventPhase);
  console.log(event.currentTarget);
  console.log(event.target);
});
parent2.addEventListener("click", function (event) {
  console.log("parent2 got event!");
  console.log(event.eventPhase);
  console.log(event.currentTarget);
  console.log(event.target);
});
```

addEventListner의 세 번째 인자를 전달하지 않으면
기본으로 Bubbling phase에서 이벤트를 캐치





이벤트 propagation

```
btn.addEventListener("click", function (event) {  
  console.log("button got event!");  
});  
  
parent2.addEventListener("click", function (event) {  
  console.log("parent2 got event!");  
});
```

button got event!

parent2 got event!

← 원하지 않는
경우라면?

버튼 클릭 시 버블링으로 인해
의도하지 않은 부모의 핸들러까지 실행될 수 있다.



이벤트 propagation

```
btn.addEventListener("click", function (event) {  
  event.stopPropagation();  
  console.log("button got event!");  
});  
  
parent2.addEventListener("click", function (event) {  
  console.log("parent2 got event!");  
});
```



button got event!

event.stopPropagation()
을 호출하면 해당 요소의
핸들러는 실행되지만,
이후 부모로의 전파는 중단된다.